AROMA

AI-powered Repository & Object Model Analyzer

Project Overview & Pitch

From idea to impact in 48 hours

IBM Bob Hackathon 2026 - Turn idea into impact faster

Built with IBM Bob IDE | Powered by IBM watsonx.ai Granite

The Problem

Developers spend 30-50% of their time understanding code rather than building. Legacy codebases are painful: no documentation, hidden dependencies, no test coverage, security blind spots, and constant context switching.

Our Solution: AROMA

AROMA (AI-powered Repository & Object Model Analyzer) is a comprehensive web-based development tool that helps developers understand, maintain, and improve legacy codebases using AI. It addresses ALL five pain points in a single, unified web application.

Hackathon Alignment

Challenge:	Turn idea into impact faster
Speed:	Get up to speed on existing code quickly
Docs:	Generate documentation and tests automatically
Effort:	Reduce effort spent on repetitive tasks

IBM Technology Stack

AI Inference:	IBM watsonx.ai Granite 4-h-small
Auth:	IBM IAM bearer token (auto-refresh)
Dev Partner:	IBM Bob IDE / Bob Shell (5 sessions)
Framework:	Next.js 15 + React 19 + TypeScript 5

Key Design Decisions

- Web application - accessible anywhere, no install
- 6 specialized modules - deep functionality per task
- Manual code input - privacy-preserving
- AI-first with fallbacks - always responsive
- IBM watsonx.ai backend - official IBM AI platform

AROMA

AI-powered Repository & Object Model Analyzer

Architecture Deep Dive

Full-stack TypeScript with IBM watsonx.ai

IBM Bob Hackathon 2026 - Turn idea into impact faster

Built with IBM Bob IDE | Powered by IBM watsonx.ai Granite

System Architecture

AROMA is a full-stack web application with a React frontend (Next.js App Router) and five server-side API routes that call IBM watsonx.ai. All IBM credentials live exclusively on the server - never sent to the browser.

Core Components

Frontend:	Next.js 15 App Router + React 19 + Tailwind CSS 4
State:	Zustand 5 global state store
UI Library:	shadcn/ui (30+ accessible components)
Charts:	Recharts (6 chart types in Health Dashboard)
Backend:	5 Next.js API Routes (server-side only)
AI Client:	lib/watsonx.ts (IBM watsonx.ai REST + IAM)
Database:	Prisma ORM + SQLite

Data Flow

1. User pastes code in browser
2. Frontend calls POST /api/* with code payload
3. API route builds system prompt + user message
4. lib/watsonx.ts obtains IAM token (cached, 60-min TTL)
5. POST to IBM watsonx.ai /ml/v1/text/chat endpoint
6. Granite model returns structured JSON or Markdown
7. API route parses response, applies fallback if needed
8. Frontend renders results in UI module

Security Architecture

- All AI calls server-side only - API keys never reach browser
- .env is gitignored and excluded via .bobignore
- Input validation on all routes (code/message required)
- TypeScript strict mode enabled
- Fallback data for every route when AI is unavailable

AROMA

AI-powered Repository & Object Model Analyzer

IBM watsonx.ai Integration

Granite foundation model inference via REST API

IBM Bob Hackathon 2026 - Turn idea into impact faster

Built with IBM Bob IDE | Powered by IBM watsonx.ai Granite

Integration Overview

All five API routes share a consistent integration pattern through src/lib/watsonx.ts. The module handles IBM IAM token acquisition, caching, and automatic refresh, then calls the watsonx.ai chat completions endpoint with the Granite foundation model.

Authentication Flow

1. API route calls callWatsonx() or analyzeWithWatsonx()
2. watsonx.ts checks cached IAM token (valid 60 min)
3. If expired: POST https://iam.cloud.ibm.com/identity/token
4. Receives bearer token with expires_in: 3600
5. Token cached with 5-min refresh buffer
6. Authorized POST to watsonx.ai chat endpoint

Model Configuration Per Route

API Route	Temp	MaxTok	Rationale
/api/analyze	0.1	2000	Deterministic JSON extraction
/api/chat	0.3	1500	Slightly creative explanations
/api/refactor	0.1	2000	Consistent pattern detection
/api/generate	0.4	3000	More creative for docs prose
/api/security	0.05	2000	Highly deterministic findings

Response Parsing

watsonx.ts handles two response formats:
1. Chat completions: choices[0].message.content (current)
2. Text generation: results[0].generated_text (legacy)

Two JSON extraction helpers:
- extractJSON: Strips markdown fences, extracts JSON
- safeParseJSON: Parses JSON with fence stripping

All routes have static fallback data when AI is unavailable.

Verified Live Test Results

All 5 API routes tested with live IBM watsonx.ai:
- /api/analyze: Files, architecture, complexity OK
- /api/chat: 1245-char markdown flow analysis OK
- /api/refactor: 3 refactoring suggestions OK
- /api/generate: 3042-char README.md OK

- /api/security: SQL Injection + eval() detected, score 60/100

AROMA

AI-powered Repository & Object Model Analyzer

Six AI-Powered Modules

Each powered by IBM Granite via watsonx.ai

IBM Bob Hackathon 2026 - Turn idea into impact faster

Built with IBM Bob IDE | Powered by IBM watsonx.ai Granite

1. Code Explorer - Architecture Mapping

The Question:	What exists?
API Route:	/api/analyze
Output:	{ files, dependencies, architecture, complexity, suggestions }

Key Features:

- AI identifies files, languages, line counts
- Function extraction with complexity scores
- Dependency mapping of import chains
- Architecture summary (MVC, microservices)
- 5 AI improvement suggestions

2. Flow Tracer - Interactive Code Flow

The Question:	How does it work?
API Route:	/api/chat
Output:	{ response: markdown }

Key Features:

- Conversational chat about codebase
- Flow tracing from endpoints to databases
- Multi-turn context (last 8 messages)
- Markdown technical responses
- Architecture explanations

3. Smart Refactor - Pattern Detection

The Question:	What should I fix?
API Route:	/api/refactor
Output:	{ suggestions: [...] }

Key Features:

- Deprecated patterns (var, callback hell)
- Code smell detection (magic numbers)
- Severity: Critical/High/Medium/Low
- Code diff with one-click copy
- Modern alternatives

4. Doc & Test Generator - Documentation Engine

The Question:	How do I document?
API Route:	/api/generate
Output:	{ title, doc, language }

Key Features:

- README.md with architecture + API ref
- API Documentation with schemas
- Vitest test suite with mocks
- JSDoc/TSDoc annotations
- 4 generation types

5. Health Dashboard - Quality Metrics

The Question:	How healthy is my code?
API Route:	(state-driven)
Output:	6 Recharts visualizations

Key Features:

- Health trend over 8 weeks
- Language distribution (Pie)
- Module complexity (Bar)
- Quality radar 6-axis
- AI-generated insights

6. Security Scanner - Vulnerability Detection

The Question:	Is my code safe?
API Route:	/api/security
Output:	{ findings: [...], score }

Key Features:

- OWASP Top 10 (10/10)
- SQL Injection (CWE-89), XSS (CWE-79)
- Hardcoded secrets (CWE-798)
- eval() injection (CWE-95)
- Score 0-100

AROMA

AI-powered Repository & Object Model Analyzer

IBM Bob IDE Development

Bob Shell sessions that built AROMA

IBM Bob Hackathon 2026 - Turn idea into impact faster

Built with IBM Bob IDE | Powered by IBM watsonx.ai Granite

IBM Bob as Development Partner

IBM Bob served as the primary AI development partner throughout the AROMA build. Bob Shell was used via CLI for all five development sessions, providing code review, architecture analysis, and verification of the IBM watsonx.ai integration.

Session Summary

#	Session	Focus	Bob Mode
1	Project Init	Full project review	Ask + Code review
2	watsonx Integration	5 API routes, Granite	Code review
3	Security Scanner	OWASP Top 10, CWE	Security audit
4	Smart Refactor	Pattern detection	Refactoring
5	Final Review	Full audit	Review + cleanup

Session Reports in Repository

All five Bob Shell task session reports are stored in bob_sessions/ directory. Each folder contains:
- task-history.md: Full exported task history from Bob Shell
- screenshot.png: Task session consumption summary

Session Details

Session 1: Project Initialization
Bob reviewed the entire AROMA project, identified all 6 modules, and confirmed the IBM watsonx.ai integration architecture. Verified Granite model config and IAM token management.

Session 2: watsonx.ai Integration
Bob reviewed all 5 API routes and verified watsonx.ai integration. Confirmed temperature settings, JSON extraction, and fallback strategies for each route.

Session 3: Security Scanner
Bob reviewed the Security Scanner component and API route, verifying OWASP Top 10 detection with CWE references. Confirmed security score calculation.

Session 4: Smart Refactor
Bob reviewed the Smart Refactor module, suggested improvements to pattern detection. Verified severity classification and modern alternatives.

Session 5: Final Code Review
Bob performed comprehensive final review, checking for non-IBM AI references. Verified all .env credentials gitignored, no API keys exposed.

How AROMA Embodies the Hackathon Theme

IBM Bob Capability Used	AROMA Module
Explained existing code logic	Code Explorer
Traced data flows across files	Flow Tracer
Detected anti-patterns	Smart Refactor
Generated documentation	Doc & Test Generator
Identified security issues	Security Scanner